

C form / snapshot local vars / params in function static/global object p = malloc/calloc/realloc(...) char *p = "abc"; char a[] = "abc"; char z = p[1]; char *p = &obj; ! *pp = malloc(sizeof(* *)); *pp = &obj; *pp rule	where storage is stack frame static/global segment = heap block returned; pointer variable itself stays where declared p variable on stack/global; literal in read-only program data array contents stored in local/global object location z in local/global char object p stores address of obj heap stores one !* ; *pp is address of obj variable location ≠ pointed-to location ≠ pointed-to value	meaning / trap lifetime ends when function returns; never return pointer to local. whole-program lifetime: static local scope stays local but value persists. separate pointer storage from pointer storage; free heap block once. do not modify literal through p. local array copy on stack; writable if local char array. copies value; z is not a pointer. pointee location is wherever obj lives. **pp is object value after two dereferences. draw three boxes: pointer variable, pointee object, value inside pointee.
---	--	---

Model	Visual rule	Use in question	prototype / syntax	behavior + return/output	trap
Protocol	Draw memory/process/fd/thread picture -> owners -> edge cases -> pseudo/comments -> code -> return checks -> cleanup	Test empty/one/head/tail/middle/error; save next before free.	int pipe(int p[2]); int dup(int oldfd); int dup2(int oldfd,int newfd); FILE *fopen(int fd,const char *mode); int fcntl(FILE *stream); popen/pclose	Creates byte channel; returns 0/-1; fills read/write fds. Duplicates fd to lowest free slot; returns new fd/-1. Makes newfd refer to oldfd; returns newfd/-1. Wraps fd as stream; returns FILE*/NULL. Gets fd from stream; returns fd/-1. See stdio; command stream + wait close.	p[0] read, p[1] write; close ends same open file desc. closes newfd first; before exec fclose closes fd unfiring remains not two-way
argc/argv	arg=argument count; argv is array of C strings; argv[0] program name; argv[argc]=NULL.	Validate count before indexing; parse numbers with strtol.			
Pointers	On 64-bit, an address/pointer is usually 8B; pointee type size does not change pointer size.	sizeof(int*)=sizeof(struct big*) usually; pointer arithmetic uses pointed size. sizeof array = sizeof ptr; pass length.	sizeof a[] is object size. sizeof a[] pointer size; free pointees. arr[] may be NULL; always check before ->. array order ≠ linked order; do not use arr[i+1] unless problem says so.		
Pointer size	Array owns contiguous storage; expression often decays; parameter T a[] becomes T *a.	strncpy ending '\0'; capacity != length; strlen excludes terminator.	strncpy may not NULL; strncpy count; == address. Never return pointer to local.		
Array/ptr	strut T a[] = contiguous objects; a[] jumps by sizeof(struct T).	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Struct array	strut T a[] = contiguous objects; a[] jumps by sizeof(struct T).	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Ptr array	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Array of pointers	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Pointer graph	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
String	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Storage	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
static local	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Ownership	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
FILE* vs fd	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Proc/thread	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Mutex/CS	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Condvar	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		
Deadlock	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.	strut T a[] = contiguous pointers; pointees scattered.		

<stdio.h> streams / formatted I/O

prototype / syntax FILE *fp; FILE *stdin,*stdout,*stderr; int printf(const char *fmt,...); int fprintf(FILE *a,const char *fmt,...); int scanf(const char *fmt,...); int fscanf(FILE *a,const char *fmt,...); size_t fread(const *ptr,size_t size,size_t nmem,FILE *stream); int getc(char c); int putc(char c); char *fgets(char *a,int size,FILE *stream); FILE *fopen(const char *path,const char *mode); int fclose(FILE *stream); int feof(FILE *a); int ferror(FILE *a); int fflush(FILE *stream); FILE *fdopen(const char *nd,const char *type); int pclose(FILE *stream);	behavior + return/output Opaque buffered stream handle used by stdio calls; returned by fopen/fdopen/popen. Standard streams available at process start. Formats to stdout; returns chars printed or negative. Formats to, closes stream; returns char or negative. Reads stdin into addresses; returns assigned count or EOF. Reads stream into addresses; returns assigned count or EOF. Reads binary items; returns complete frame read. Reads one char from stdin; returns char as int or EOF. Writes one char to stdout; returns char or EOF. Reads at most size-1 char; NULL terminates; returns s or NULL. Pushes path as stream; returns FILE* or NULL. Flushes/closes stream; returns 0 or EOF. Suspect EOF/error flags after an attempted read. Flushes output stream; returns 0 or EOF. Runs shell cmd with one stream; returns FILE* or NULL. Closes popen stream and waits; returns wait status or -1.	trap not fd stdout buffered; stderr diag format/arg types check stream need cleanup with for fs loop exact count check read store in int EOF means error keeps vs. size includes NUL "r" truncates stream invalid after not loop condition input flush nonportable shell, one-way not fdsize
---	--	---

<string.h> / <ctype.h>

prototype / syntax size_t strlen(const char *s); size_t strnlen(const char *s,size_t n); char *strcpy(char *dst,const char *src); char *strncpy(char *dst,const char *src,size_t n); int strcmp(const char *a,const char *b); int strncmp(const char *a,const char *b,size_t n); char *strerror(int errnum); int memcmp(const void *p,const void *q,size_t n); int toupper(int c);	behavior + return/output Counts chars before first NUL. Counts up to max or NULL. Copies s including NUL; returns dst. Copies up to n; pads if short; returns dst. Lexicographic compare: 0 equal, <0/>0 differ. Compare at most n chars; 0 means equal prefix. Readonly-through-expression / no-alias promise. Compares n raw bytes; 0 equal. Converts lower-case char; accepts EOF or unsigned-char value.	trap not capacity not new readable max bytes dst must fit may not terminate == compares addr not full equality static/reused text not string-aware plain char may be negative
---	--	---

<stdlib.h> memory / conversion / sort

prototype / syntax void *malloc(size_t size); void *calloc(size_t n,size_t size); void *realloc(void *ptr,size_t size); void free(void *ptr); void qsort(void *base,size_t n,size_t size,int (*cmp)(const void*,const void*)); long int strtol(const char *s,char **end,int base); void exit(int status);	behavior + return/output Allocates uninitialized heap bytes; returns ptr or NULL. Allocates n*size zeroed bytes; returns ptr or NULL. Resizes/moves allocation; returns new ptr or NULL. Releases heap allocation; accepts NULL; no return. Sorts array using comparator; no return. Converts string; stores first unconverted char in *end. Terminates process with stdio cleanup; no return.	trap use sizeof *p overflow concern old valid on fail heap once only cmp gets elem ptrs check end+errno not child fork fail path
--	--	--

C language / declarations

prototype / syntax sizeof type or sizeof(type) NULL typedef old new; enum E {A,B}; struct S {...}; union U {...}; unsigned f3; ret (f)(args) const, restrict static, extern	behavior + return/output Compile-time byte size result of type size_t. Null pointer constant for pointer context. Creates type alias. Named integer constants. Groups fields; use . for object, -> for pointer. Members share storage; one active interpretation. Bitfield with implementation-defined layout. Function pointer matching signature. Readonly-through-expression / no-alias promise. Internal linkage/persistent local; external declaration.	trap contray vs ptr not '\0' not new runtime type default increments padding/alignment needs tag not portable disk format parentheses matter not ownership: UB if lie scope changes meaning; one definition
---	---	--

Compiler / tools

prototype / syntax gcc file.c -o prog gcc -c file.c -o file.o gcc a.o b.o -o prog -Wall -Wextra -Werror -g -g3 -fsanitize=address gdb ./prog; valgrind ./prog make; diff; scripts man 1/2/3 name objdump; readelf	behavior + return/output Compile/link to executable or diagnostics. Compile object only: unresolved external refs may remain. Link objects / libraries. Enable warnings; make warnings fatal. Debug symbols; low optimisation. Adds ASan runtime checks; reports on execution. Debug state / dynamic memory diagnostics. Automate build and regression tests. Flags for commands / syscalls / library calls. Inspect object/executable assembly, symbols, ELF metadata.	trap run correct output not linked undefined/duplicate refs fix warnings for gdb clarity must run program compile with -g tabs/quoting read flags - returns platform detail
---	--	--

<errno.h> errors

prototype / syntax extern int errno; EINTR errno/errno_t/errno errno	behavior + return/output Holds detail after many failed calls. Call interrupted by signal. Nonblocking operation would block. Range error, e.g. strtol.	trap not errno read only after failure retry depends context normal state set errno=0 before
--	--	--

<fcntl.h>/<unistd.h> fd I/O

prototype / syntax int open(const char *path,int flags,...); size_t read(int fd,void *buf,size_t count); size_t write(int fd,const void *buf,size_t count); int close(int fd); int fcntl(int fd,int cmd,...); O_RDONLY/O_WRONLY/O_CREAT/O_TRUNC/O_APPEND STDIN_FILENO/STDOUT_FILENO/STDERR_FILENO	behavior + return/output Opens path; returns fd >=0 or -1. Reads up to count; >0 bytes, 0 EOF, -1 error. Writes up to count; returns bytes or -1. Closes descriptor; returns 0 or -1. Gets/set descriptor flags/control; returns varies or -1. Open/fcntl flags selecting access/create/truncate/nonblock. Raw fd constants 0/1/2.	trap mode with O_CREAT partial, EINTR partial write may reveal error preserve flags nonblock gives EAGAIN not FILE streams
--	--	--

Processes: <unistd.h>/<sys/wait.h>/<spawn.h>

prototype / syntax pid_t pid; pid_t fork(void); int exec(const char *path,const char *arg0,...,NULL); int execl(const char *file,const char *arg0,...,NULL); int execv(const char *path,char *const argv[]); int execlp(const char *file,char *const argv[]); int posix_spawn(pid_t *pid,const char *path,...,char *const envp[]); pid_t wait(int *status); waitpid(pid,status,opt); waitpid(pid,status,opt); void _exit(int status);	behavior + return/output Type for process IDs. Duplicate process; returns 0 child, child pid parent, -1 fail. Replace image by path; success no return, fail -1. Like exec but PATH search. Exec path using NULL-ended argv; no return on success. Exec PATH command using argv; no return or -1. Spawn child; returns 0 or error number. Reaps child; returns pid, 0 WNOHANG, or -1. Wait normal exit then extract code. Exit immediately; no stdio cleanup; no return.	trap unique-inh re-owns twice NULL end PATH + NULL argv termination after call = fail path spawn itself error avoid zombies check with wait child after exec fail
---	---	--

IPC / fd bridge

Boxed code skeletons: keep context + cleanup paths

input pipeline: scanf + fgets + getchar EOF

```
/* scanf then fgets: remove leftover newline first */
if (scanf("%d", &n) != 1) goto bad_input;
int ch; while ((ch=getchar())!='\n' && ch!=EOF) { }
char line[N];
if (!fgets(line, sizeof line, stdin)) goto eof_or_error;
line[strlen(line,"\\n")] = 0; /* strip newline */
/* raw char loop must use int, not char */
for (int c; (c=getchar()) != EOF; ) putchar(c);
```

strtol: complete validation template

```
errno = 0;
char *end = NULL;
long v = strtol(argv[1], &end, 10);
if (end == argv[1]) goto no_digits;
if (*end == '\\0') goto trailing_junk;
if (errno == ERANGE) goto out_of_range;
/* now v is valid; then range-check for target type */
```

allocation + safe realloc context

```
size_t cap = 16, len = 0;
T *a = malloc(cap * sizeof *a);
if (!a) goto fail;
if (len == cap) {
    size_t newcap = cap * 2;
    T *tmp = realloc(a, newcap * sizeof *a);
    if (!tmp) goto fail; /* a still valid */
    a = tmp; cap = newcap;
}
```

linked list: head / tail / middle insert

```
/* head insert */
new->next = head; head = new;

/* tail insert */
new->next = NULL;
if (!head) head = new;
else { cur=head; while (cur->next) cur=cur->next; cur->next=new; }

/* insert after prev: require prev != NULL */
new->next = prev->next; prev->next = new;
```

linked list: delete via pointer-to-link

```
struct node *link = &head;
while (*link && (*link)->v != key) link = &(*link)->next;
if (*link) {
    struct node *old = *link;
    *link = old->next; /* bypass before free */
    free(old);
}
```

linked list: free whole list

```
while (head) {
    struct node *next = head->next;
    free(head);
    head = next;
}
```

array of pointers / pointer graph

```
for (int i=0; i<n; i++) {
    T *curr = arr[i]; /* arr[i] is T */
    if (!curr) continue; /* check before -> */
    T *front = curr->next; /* linked graph */
    use(curr->field);
}
/* array order != linked order; do not assume arr[i+1]. */
```

fread records + strtok tokenize

```
struct rec rec;
while (fread(&rec, sizeof rec, 1, fp) == 1) use(rec);
if (ferror(fp)) goto ioerr; /* short ffinal read may be EOF */

for (char *tok= strtok(line, "");) tok; tok= strtok(NULL, "") {
    /* tok points inside line; strtok modifies line */
}
```

frequency array + lookup / parallel arrays

```
int freq[256] = {0};
for (int c; (c=getchar()) != EOF; )
    freq[(unsigned char)c]++;

const char *names[] = {"red", "green"};
int vals[] = {1, 2};
for (size_t i=0; i<n; i++)
    if (strcmp(key, names[i]) == 0) return vals[i];
```

qsort comparator + signal flag

```
int cmp(const void *pa, const void *pb) {
    const T *a = pa, *b = pb; /* pa/pb point to elements */
    return (a->k > b->k) - (a->k < b->k);
}

static volatile sig_atomic_t stop;
static void h(int s) { (void)s; stop = 1; }
struct sigaction sa = {0};
sa.sa_handler = h; sigaction(SIGINT, &sa, NULL);
```

robust read/write: partial + EINTR

```
for (;;) {
    ssize_t n = read(in, buf, sizeof buf);
    if (n == 0) break; /* EOF */
    if (n < 0) { if (errno == EINTR) continue; goto fail; }
    for (ssize_t off = 0; off < n; ) {
        ssize_t v = write(out, buf + off, n - off);
        if (v < 0) { if (errno == EINTR) continue; goto fail; }
        off += v; /* partial writes allowed */
    }
}
```

fork + execvp + waitpid

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); goto fail; }
if (pid == 0) {
    execvp(av[0], av);
    perror(av[0]); /* exec success never returns */
    _exit(127);
}

int st; waitpid(pid, &st, 0);
if (WIFEXITED(st)) code = WEXITSTATUS(st);
```

pipe + dup2: child stdout -> parent

```
int pfd[2]; pipe(pfd);
if (fork()==0) {
    dup2(pfd[1], STDOUT_FILENO);
    close(pfd[0]); close(pfd[1]);
    execvp(cmd[0], cmd); _exit(127);
}
close(pfd[1]); /* parent reads pfd[0] */
/* read until EOF, close pfd[0], wait child */
```

anonymous pipe model: parent-child IPC

```
int p2c[2], c2p[2];
pipe(p2c); pipe(c2p); /* each pipe is one-way */
/* parent: write p2c[1], read c2p[0]
   child: read p2c[0], write c2p[1]
   after fork: each process closes every unused end.
   no exec needed unless running a different program. */
```

linear process chain creation

```
/* process chain P0 -> P1 -> ... -> Pn-1 */
for (int i=0; i<n; i++) {
    /* create pipes between Pi and Pi+1, then fork */
    pid_t pid = fork();
    if (pid < 0) goto fail;
    if (pid == 0) { /* child becomes Pi+1 */
        my_id = i + 1;
        /* keep parent pipe ends; close unrelated ends */
        continue; /* child may create next link */
    }
    child_pid = pid; /* parent keeps child link */
    break;
}
```

routing protocol in process chain

```
struct msg { int src, dst, len; char msg[50]; };
if (m.dst == my_id) deliver(m);
else if (m.dst < my_id) write(parent_w, &m, sizeof m);
else write(child_w, &m, sizeof m);
/* proof: distance=abs(my_id-dst); every forward reduces
   distance by 1 -> no cycle; destination receives exactly once. */
```

cleanup / SIGTERM discipline

```
/* handler only sets flag; main loop performs cleanup */
static volatile sig_atomic_t stop;
static void term(int s){ (void)s; stop = 1; }

/* cleanup path */
close_owned_pipe_fds();
free(state);
if (child_pid > 0) { kill(child_pid, SIGTERM); waitpid(child_pid, NULL, 0); }
```

popen + mmap bytes

```
FILE *fp = popen(cmd, "r");
if (!fp) goto fail;
while (!fgets(buf, sizeof buf, fp)) use(buf);
int st = pclose(fp);

struct stat sb;
if (statat(fd, &sb) == 0) {
    char *m = mmap(NULL, sb.st_size, PROT_READ, MAP_PRIVATE, fd, 0);
    if (m != MAP_FAILED) { scan(m, sb.st_size); munmap(m, sb.st_size); }
}
```

select / poll readiness

```
fd_set r; FD_ZERO(&r); FD_SET(fd, &r);
if (select(fd+1, &r, NULL, NULL, NULL) > 0 && FD_ISSET(fd, &r))
    read(fd, buf, sizeof buf);

struct pollfd fds[] = {fd, POLLIN, 0};
if (poll(fds, 1, -1) > 0 && (fds[0].revents & POLLIN))
    read(fd, buf, sizeof buf);
```

pthread args + mutex / condvar

```
struct arg args[N]; pthread_t t[N];
for (int i=0; i<N; i++) { args[i].id=i; pthread_create(&t[i], 0, work, &args[i]); }
for (int i=0; i<N; i++) pthread_join(t[i], 0);

pthread_mutex_lock(&m);
while (!pred) pthread_cond_wait(&cv, &m);
/* use/update protected state */
pthread_mutex_unlock(&m);
/* producer: lock; change pred/state; signal/broadcast; unlock */
```

semaphore / barrier / Amdahl

```
sem_wait(&s); /* wait resource/event */
/* use resource */
sem_post(&s); /* release/signal */
phase1(); pthread_barrier_wait(&b); phase2();
S(N) = 1 / ((1-P) + P/N); max = 1/(1-P);
```

Challenge checklists	
Pointer/list	boxes/arrows; owner? caller pointer? save next? empty/head/tail/middle? array vs linked order?
String/input	cap vs len? terminator/newline? scanf/fgets? bounded copy? strtol validated?
File/IO	FILE* or fd? return? buffering? close path? binary count vs bytes?
Low I/O	partial? EINTR? nonblock? loop? errno only after fail?
Process/pipeline	argc valid? process count? exec fail path? wait/reap? ends closed? EOF possible?
Threads/sync	phased state? mutex protects invariant? arg lifetime? join? cond while? lock order?